



11

COVERT CODE EXECUTION AND MISDIRECTION



Continuing [Part III](#)'s discussion of anti-reversing tactics used by malware, in this chapter we'll look at *covert code execution*, in which malware executes code in a stealthy manner, frustrating an analyst's attempts to follow its logic and code and sometimes evading debugging altogether. This can also serve the purpose of misdirecting the analyst, causing confusion and slowing down the reverse engineering process. Let's explore a few of the specific covert code execution and misdirection techniques you might encounter.

Callback Functions

Callback functions are application-defined functions that are triggered by a specific event and are used as input for other functions. For example, the Windows API function `EnumDisplayMonitors` uses callback functions to enumerate display monitors that are configured on the host. When `EnumDisplayMonitors` is called, monitors are enumerated one by one, and information about each of them (such as screen size) is

passed to the callback function. The program defines this callback function and can point it to any code it wishes.

Malware can abuse functions such as `EnumDisplayMonitors` by creating a custom callback and pointing it to malicious code, as illustrated in **Figure 11-1**. This code will then be executed by the calling function (`EnumDisplayMonitors`), which serves the purposes of obfuscating control flow (as an anti-disassembly and anti-disassembly technique) and even possibly causing the malware analyst to lose control of the malware in a debugger. This method can also confuse some automated sandboxes.

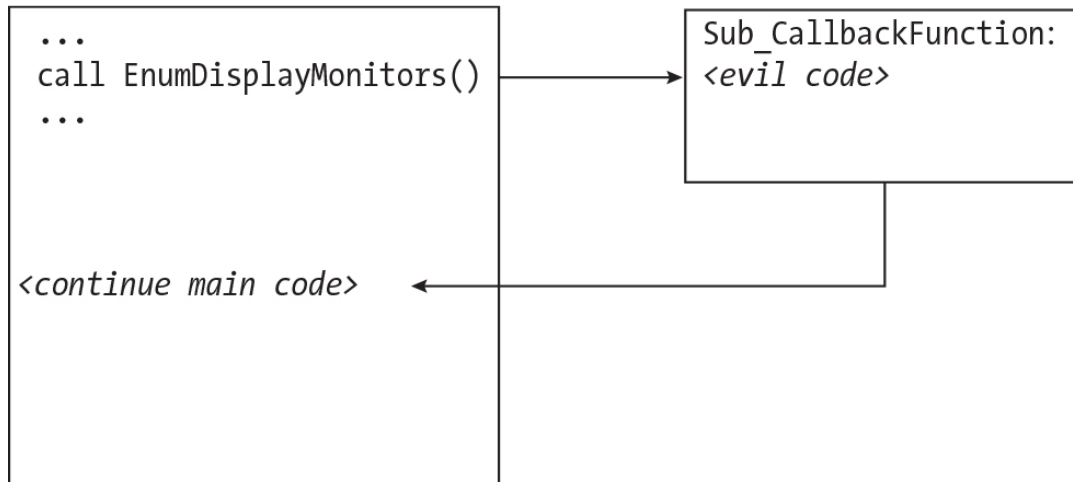


Figure 11-1: A callback function example with `EnumDisplayMonitors`

The malware in this figure calls `EnumDisplayMonitors` and defines its own callback function containing malicious code. When `EnumDisplayMonitors` is called, the control flow transfers to the malicious callback function. Seeing this behavior in a debugger or during static code analysis can be very confusing to a malware analyst who isn't aware of how callbacks work, as the jump to this callback function may not be apparent in the code.

This technique can theoretically work with nearly any Windows API function that uses callbacks (and there are several), but I've found that many of the functions that malware may abuse begin with `Enum`, such as `EnumDateFormatsEx`, `EnumSystemLanguageGroups`, and `EnumChildWindows`.

TLS Callbacks

As [Chapter 1](#) explained, a thread is a series of instructions that operates inside a process. Thread-local storage (TLS) allows each of the program's running threads to have its own version of a variable that every other thread can access with a unique value. For example, if a global variable `var` is defined in TLS, each thread in the process can store a different value in it. In this case, `var` is acting as a sort of global variable name, but with a unique value for each thread.

A TLS callback function allows programs to clear data objects from the TLS. These callback functions run before the actual program code starts, so a malware author can exploit this by crafting special TLS callback functions that execute before the main malware code begins to run. This technique not only can confuse and misdirect analysts who are debugging the malware's code but also can obfuscate the code's control flow. Let's look at a simple example of how to identify and locate TLS callback routines.

NOTE

To follow along in this section, download the sample from VirusTotal or MalShare using this hash:

SHA256:

e4bd2245b1f75abf37abd5a4b58e05f00886d56a5556080c4331847c7266b5b6

To identify malware that might be using TLS callback functions, you can use one of many static executable analyzer tools, such as PEStudio, my personal favorite. PEStudio has a tab called TLS Callbacks that lists any registered callbacks in the executable file and their addresses. Our malware file in [Figure 11-2](#), for example, contains two TLS callbacks.

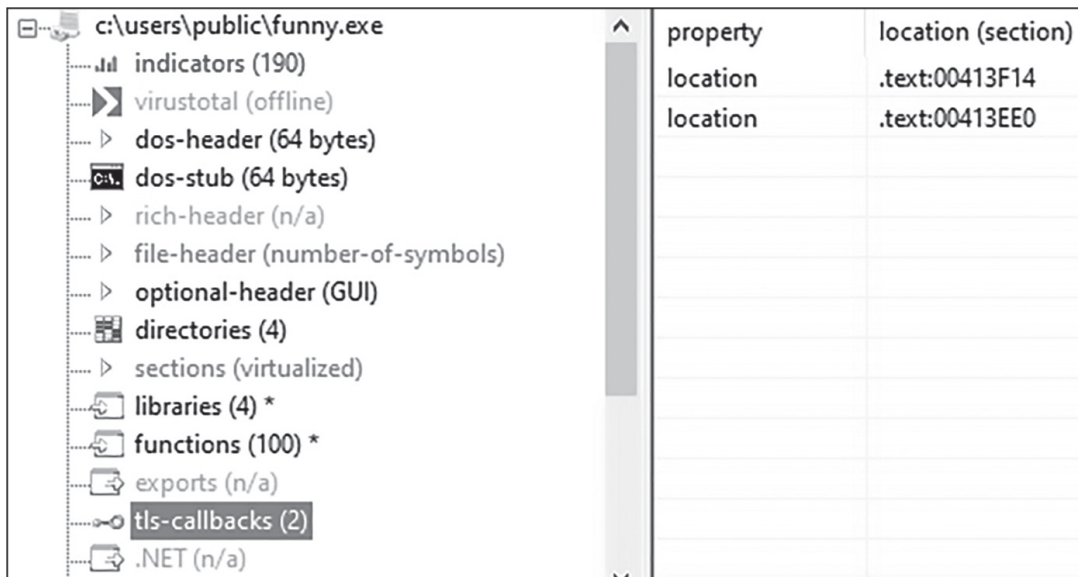


Figure 11-2: A TLS callback viewed in PESTudio

Keep in mind that TLS callbacks were originally designed for benign purposes, so the presence of one doesn't necessarily mean that the executable is malicious.

To better understand how TLS callback functions can confuse and misdirect analysts and to learn how to work with malware that is using them, let's take a look at this sample in a debugger. I'm using x64dbg, but any similar debugger should work.

First, if you spot TLS callbacks in a malware sample (using PESTudio, for example), always make sure that the debugger is configured to break on TLS callback functions. Otherwise, the debugger will execute and may not break on the callback function, and you'll likely never realize it ran. To ensure x64dbg breaks on a TLS callback, click **Options ▶ Preferences ▶ Events** and confirm that TLS Callbacks is checked, as shown in **Figure 11-3**.

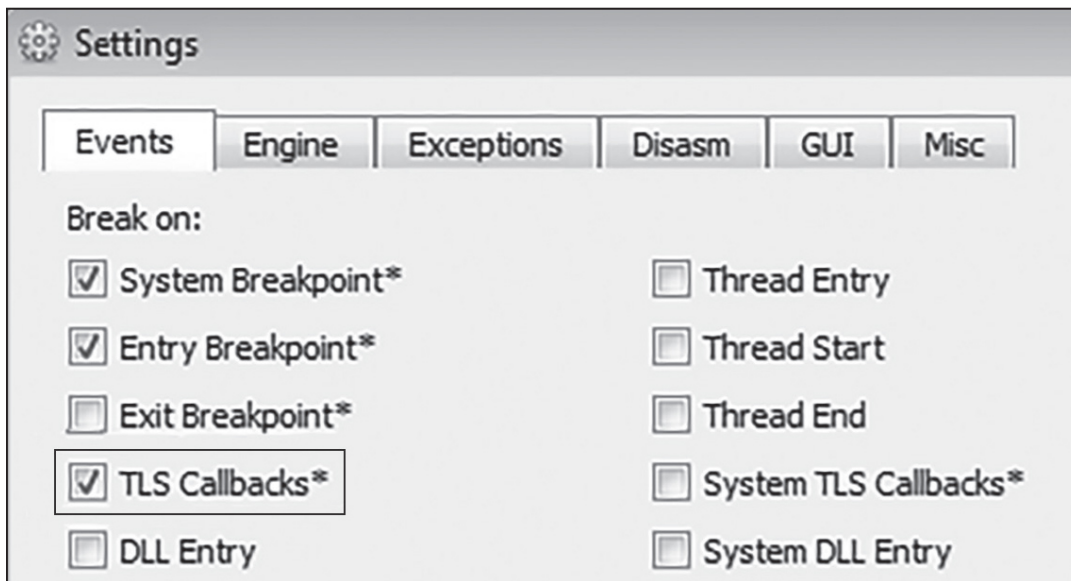


Figure 11-3: Enabling breakpoints on TLS callbacks

After attaching the malware sample to the debugger, you can run the malware sample normally by pressing F9, and the debugger will break on the TLS callback function, as shown in [Figure 11-4](#).



Figure 11-4: Hitting the TLS breakpoint

The debugger is now paused on the TLS callback function address, which will likely be the entry into the malicious code that the malware wishes to covertly execute, as shown in [Figure 11-5](#).

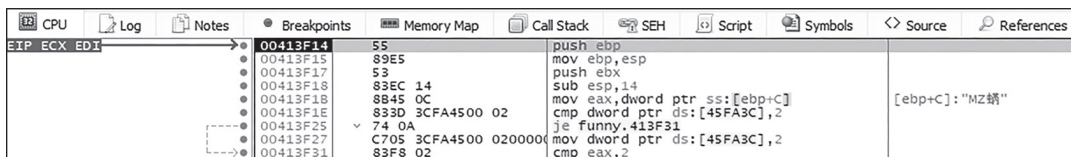


Figure 11-5: TLS callback code

Note that TLS callbacks aren't always so clearly registered and displayed in static analysis tools. TLS entries are stored in the Thread Environment Block (TEB), a data structure that stores information about the currently running thread (see [Chapter 1](#) for a refresher on it). Malware could modify its own TEB at runtime, possibly manipulat-

ing the TLS callbacks and adding or removing callbacks dynamically. In doing so, malware authors could hide their TLS callbacks so that they execute even more covertly and evade analysis tools.

Structured Exception Handling

As its name suggests, the *structured exception handler (SEH)* is how Windows applications handle exceptions. Whenever a Windows program runs into an exception, it invokes SEH. Developers can choose to execute certain code if an exception occurs in their program by implementing an SEH *record*. For example, if the program throws an error because it's missing a certain required file, the developer might instruct it to display a pop-up box with the message, "The required file does not exist!" This instruction often appears in the form of a `try... catch` or `try... except` sequence. The program will *try* to execute some code, and if the code fails for some reason, the *catch* (exception) will be executed.

SEH consists of multiple records that are stored on the program's stack. Each record in turn is composed of two addresses: the first is a pointer to the function responsible for handling the exception (that is, the *exception handler*), and the second is a pointer to the previously defined SEH record, which creates a chain of SEH records (typically called a *linked list* in Windows).

The address of the exception handler is stored in the special CPU register FS (GS for 64-bit applications), which points to the TEB. In the TEB structure, `fs:[0]` contains the current SEH frame, which points to the first SEH record on the stack. **Figure 11-6** illustrates this structure.

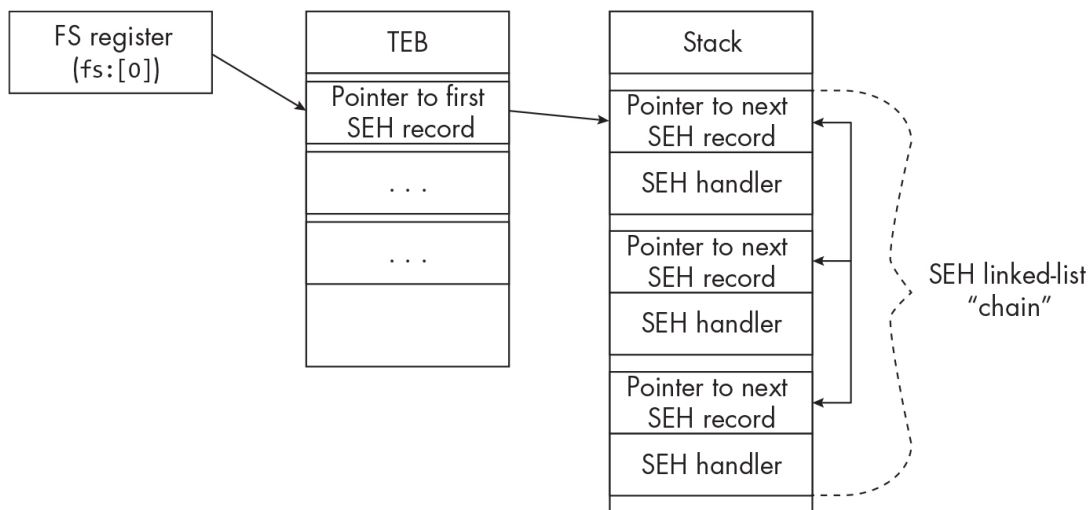


Figure 11-6: An SEH linked-list chain

Notice that the FS register points to the address of the TEB, which in turn contains a pointer to the first SEH record on the stack. The SEH record contains an address for the next SEH record in the chain, as well as an address for the exception handler (the code that will be executed when the exception is triggered).

When a program adds a new SEH record to the SEH chain on the stack, it must first push the address of the new handler to the stack and then push the address of the previous handler to the stack. This looks like the following code:

```
--snip--
push HandlerAddress ; Address of new handler
push fs:[0]         ; Address of old handler
mov fs:[0], esp
--snip--
```

After pushing the new and previous handler addresses to the stack, the instruction `mov fs:[0], esp` sets up the new handler. Once a new exception occurs in the program, `HandlerAddress` will be the “first responder” to the exception.

Just as many benign Windows features can be repurposed for malicious uses, SEH chains can be abused to obfuscate the malware’s con-

trol flow and misdirect the analysts debugging the code. Let's take a look at this in practice.

NOTE

To follow along, download the sample from *VirusTotal* or *MalShare* using this hash:

SHA256:

`d52f0647e519edcea013530a23e9e5bf871cf3bd8acb30e5c870ccc8c7b89a09`

You'll also need a debugger such as *x64dbg*.

Keep in mind that this sample is a ransomware variant, so be sure to take precautions. You might consider using some of the tips discussed in [Appendix A](#).

First, rename the file with the `.exe` extension (such as *evil.exe*) and open it in *x64dbg*. (It's a 32-bit file, so you'll need to open the file in the 32-bit version.) The executable will be paused at this point. Select **Debug ▶ Run to User Code** to skip to the beginning of the malware's code. You should now be at the malware's entry point, as shown in [Figure 11-7](#).

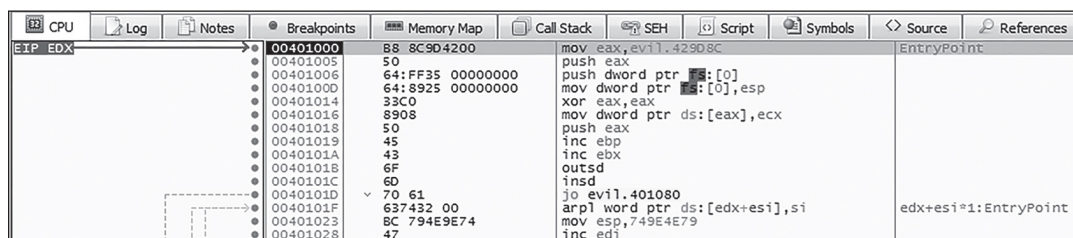


Figure 11-7: The malware's entry point in *x64dbg*

The following lines of code will be executed by the malware; note that the addresses may be different in your running sample, but the code should be similar:

```

mov  eax, evil.429D8C
push eax
push dword ptr fs:[0]
mov  dword ptr fs:[0],esp
xor  eax,eax
mov  dword ptr ds:[eax],ecx

```

First, the malware moves the address `evil.429D8C` into `eax` and pushes it to the stack. This address contains the malicious code that will be executed by the exception handler. Next, the malware pushes the current value stored in `fs:[0]` to the stack, which points to the topmost SEH record. Then, the malware moves the value of `esp` (the stack pointer) to `fs:[0]`, which effectively adds the address of the new handler.

To trigger the exception handler and stealthily execute its code, the malware must force an exception. To do this, the malware clears the `eax` register using `xor` and then executes a `mov` instruction that attempts to move the value in `ecx` into the address stored in `eax`. Because the value in `eax` is currently 0, this results in an `EXCEPTION_ACCESS_VIOLATION` error in x64dbg (shown in [Figure 11-8](#)).

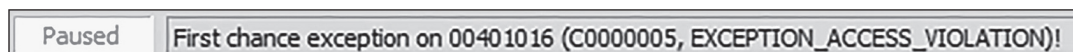


Figure 11-8: An exception forced by the malware

The control flow is transferred to the handler containing the malicious code, as shown in [Figure 11-9](#).

CPU			
Log			
Notes			
Breakpoints			
Memory Map			
Call Stack			
SEH			
Script			
Symbols			
<> Source			
References			
EIP	ECX	00429D8C	B8 118B42F0
		00429D91	8D88 9E120010
		00429D97	8941 01
		00429D9A	8B5424 04
		00429D9E	8B52 0C
		00429DA1	C602 E9
		00429DA4	83C2 05
		00429DA7	2BCA
		00429DA9	894A FC
		00429DAC	33C0
		00429DAE	C3
			mov eax,F0428B11
			lea ecx,dword ptr ds:[eax+1000129E]
			mov dword ptr ds:[ecx+1],eax
			mov edx,dword ptr ss:[esp+4]
			mov edx,dword ptr ds:[edx+c]
			mov byte ptr ds:[edx],E9
			add edx,5
			sub ecx,edx
			mov dword ptr ds:[edx-4],ecx
			xor eax,eax
			ret

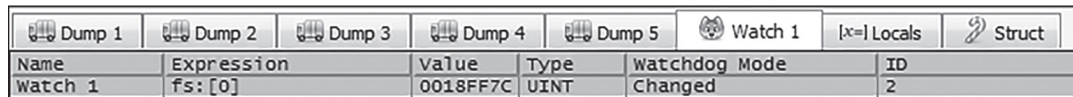
Figure 11-9: Transfer of the control flow to the handler containing malicious code

Unless we explicitly tell the debugger to break at this handler code, however, the code will be executed too quickly for us to inspect it. To address this issue, we'll set a breakpoint on this handler code by using the command `bp 00429D8C` in the debugger. Now, if we continue executing the sample, we'll break on the malicious handler code, as shown in [Figure 11-9](#). If you're already past this point in the debugger, you may need to terminate the sample and rerun it, being sure to set the breakpoint on `00429D8C`.

Had we not known that the malware was using this SEH abuse technique and had we not closely inspected the code, we likely would have completely missed this code execution. Since many disassemblers may not be SEH aware, we also would not have seen this jump when performing static code analysis.

Also, rather than directly altering the SEH chain, malware can abuse SEH by using the `KiUserExceptionDispatcher` function, which takes an exception handler address as a parameter. Malware can pass an arbitrary address to this function, adding a new handler to the current SEH chain. Then, by forcing an exception as just described, the malware can covertly execute malicious code.

When you're analyzing malware that uses the techniques discussed here, it helps to monitor the SEH chain. There are two ways to do this. In x64dbg, you'll see an SEH tab that lists the SEH chain. In my experience, however, this feature isn't always reliable. The better option is to add a *watch* for `fs:[0]` modifications. This way, you'll be alerted when malware manipulates the data stored in a register or address such as `fs:[0]`. To do this in x64dbg, navigate to the **Watch** tab underneath the CPU window, right-click and select **Add**, and then type `fs:[0]` (see [Figure 11-10](#)).



Name	Expression	Value	Type	Watchdog Mode	ID
Watch 1	fs:[0]	0018FF7C	UINT	Changed	2

Figure 11-10: Adding a watch expression in x64dbg

Once you've added the watch, right-click it and select **Changed** on the menu. This will pause the program every time the value in `fs:[0]` changes, as shown in [Figure 11-11](#).

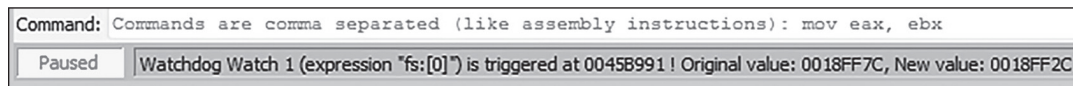


Figure 11-11: Triggering a watch expression

VEH and 64-Bit SEH

Another important exception-handling mechanism in Windows is *vector exception handling (VEH)*, an extension to SEH for Windows x86 applications. VEH is sometimes used alongside SEH in 32-bit Windows applications but takes precedence over it; that is, if an exception is triggered in the application, VEH will attempt to handle it before SEH does.

Like SEH, VEH can be abused by malware. For example, malware can call the Windows API function `AddVectoredExceptionHandler` or `RtlAddVectoredExceptionHandler`, both of which will register a new VEH record. These functions accept a parameter called `PVECTORED_EXCEPTION_HANDLER`, which represents the function that will be called once the exception occurs. The malware can then purposefully trigger an exception, in which case VEH will be triggered and the malicious code will be executed. If you happen to spot malware calling these functions, it's worth taking a deeper look to see if it's abusing VEH to covertly execute code.

In x64 applications, SEH (sometimes called *x64 SEH*) is implemented as a table that is stored in the executable's PE header. This table contains descriptions for all of the program's exception handling. Furthermore, x64 SEH uses a technique known as *stack unwinding*, which is executed in kernel mode and involves popping addresses off the stack to resume execution at another address on the stack. Stack unwinding is beyond the scope of this book, but exploits are uncom-

mon since the x64 SEH table is stored in the file header, making it difficult to tamper with the SEH chains, and because the stack unwinding takes place in kernel mode, whereas most malware runs in user mode. It's also important to note that VEH can be implemented in 64-bit applications, so the VEH abuse scenario applies for x64-bit malware as well.

Hidden Threads

The Windows API exposes a function called `NtSetInformationThread` that can be used to set the priority of a thread. `NtSetInformationThread` has a parameter called `THREADINFOCLASS`, or `ThreadInformationClass`, which points to a structure that may contain several values, one of which is particularly interesting: `ThreadHideFromDebugger`. If this value is set, the code thread will no longer send debug events to the debugger, meaning the code will be essentially flying under the radar. This presents an opportunity for malware to bypass any debuggers that an analyst may have attached and to covertly execute its code. The simplest way to overcome this technique is to look for calls to `NtSetInformationThread` and set breakpoints on them. Once a breakpoint is hit, modify the parameters to the function or simply patch the function call out of the code.

A similar evasion technique involves the `NtCreateThreadEx` function, which has a special flag, `THREAD_CREATE_FLAGS_HIDE_FROM_DEBUGGER`, that can be set to hide the newly created thread from the debugger. As you might guess, this can cause problems for malware analysts, as the code will be executed outside the immediate scope of the debugger. Be on the lookout for malware invoking the `NtCreateThreadEx` function with the `THREAD_CREATE_FLAGS_HIDE_FROM_DEBUGGER` flag (0x4 in hex) enabled.

PROCESS INJECTION

One final covert code execution and misdirection technique worth mentioning is *process injection*, which allows malware to write (or “inject”) code into

the calling process or a remote process and run the code in that process's context. This can cause headaches for a malware analyst attempting to follow a malware sample's execution in a sandbox or investigating its code in a debugger. Since process injection can also be used for evading host defenses and during unpacking, we'll cover it more thoroughly in [Chapters 12](#) and [17](#).

Summary

In this chapter, you learned about several ways in which malware can execute code while flying completely under the radar of analysis tools like debuggers. You also saw how malware can use callback functions to obfuscate its control flow while stealthily executing its malicious code. These techniques abuse underlying and otherwise legitimate Windows functions, and even though some are many years old, they're still seen in the wild. In the next chapter, we'll explore a few additional techniques that malware authors may use to covertly execute code and accomplish other evil things: process injection, manipulation, and hooking.